MILESTONE 1 - Mesh Baseline on KITTI-360

RayLiDAR

Adaptive GPU LiDAR Simulation with Hybrid Mesh + 2D Gaussians

Sizhe Liu · Xuan Zhu · Jing Huang

From street photos to simulated point clouds

JING

Real data

KITTI-360, 60 frames: photos, LiDAR, poses

JING

Mesh reconstruction

COLMAP + OpenMVS, 169k triangles

SIZHE

GPU ray tracing

Vulkan , BLAS/TLAS, ray query tracer

XUAN

LiDAR simulation

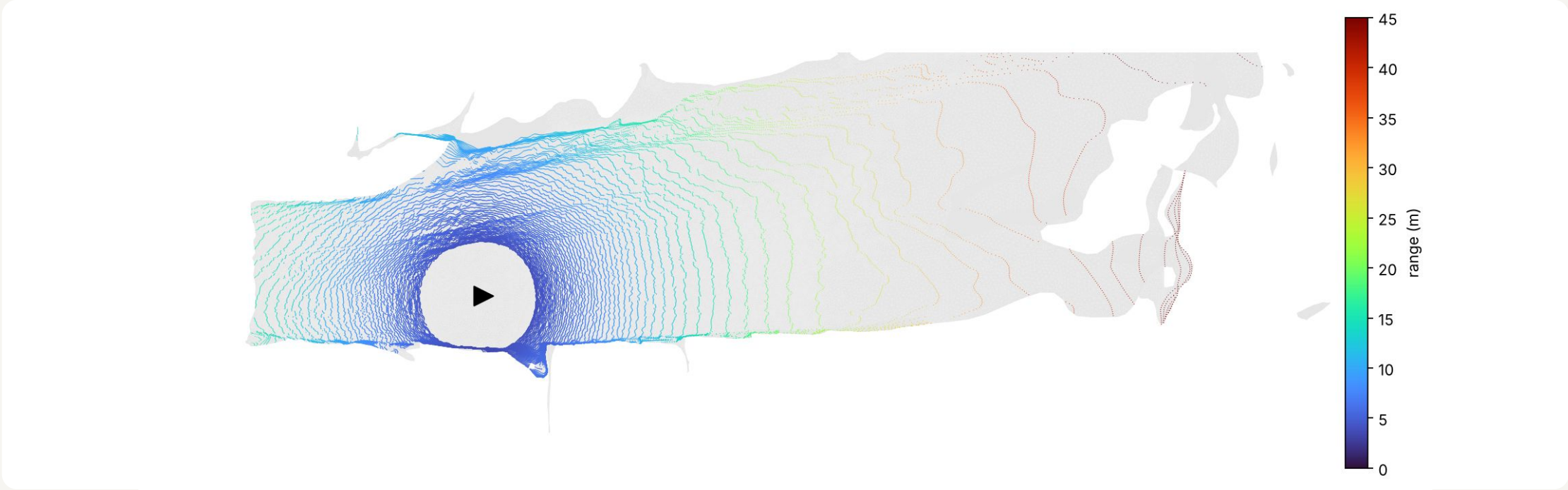
HDL-64E: 64 × 2048 rays, point cloud, timing

Then we compare every simulated range with the **real LiDAR**, ray by ray.

Layer 3	Simulation	Scene, LiDARSensor, LiDARSimulator
Layer 2	Hardware RT	AccelerationStructure, ASBuilder, RayQueryTracer
Layer 1	Vulkan Backend	VulkanContext, Buffer, ComputePipeline, CommandContext
Layer 0	Foundations	Vulkan-Hpp (vk::raii), Vulkan Memory Allocator (VMA)

Our First Simulated LiDAR Scan

We Can Now Simulate LiDAR Scans on KITTI-360



131,072 rays per frame

~96k hits per frame, 60 frames

Matches a CPU reference **ray for ray**

Accuracy

Compared with 1.77M real LiDAR rays

91.4%
of rays hit the mesh

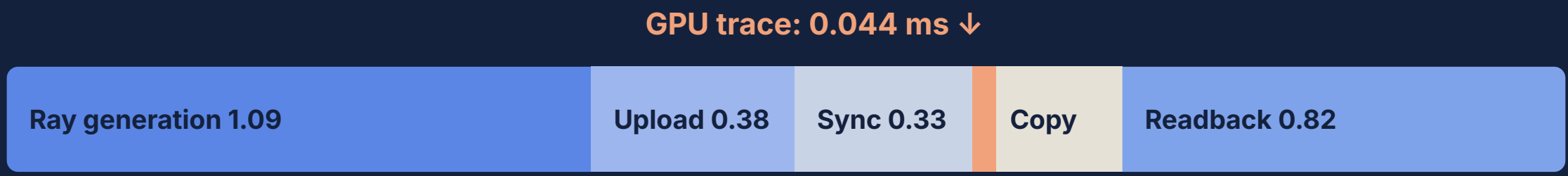
0.37 m
median range error

Frames	Mean error	Median |error|
4618–4620	≈ 0 m	0.11–0.17 m
4640–4647	−0.66 m	0.29 m
4653–4674	−3.4 to −4.2 m	0.5–0.9 m

Errors grow where the **reconstruction** is weak: plain-colored buildings and vegetation. The mesh blocks rays too early.

Performance

GPU tracing is only 1.5% of a frame



Milliseconds per HDL-64E frame (131k rays). Total 2.9 ms.

343

scans per second, end to end

4.4 G

rays per second on the GPU

Bottleneck = CPU work and data movement

Why We’re Switching from 3D to 2D Gaussians

3DGS vs. 2DGS

3D Gaussian: Where is the hit?

The ray passes through a continuous density distribution, not an explicit surface.

2D Gaussian: A natural hit position

The ray intersects the Gaussian's supporting plane at a well-defined location.

Metric	3DGS	2DGS
Depth RMSE ↓	3.6716	3.4671
Chamfer Distance ↓	0.3553	0.1077
FPS ↑	29	42

Source: LiDAR-RT supplementary material, Table 3.

Milestone 2

Mesh, Gaussian and Hybrid in one simulator

Sizhe	Gaussian ray hit: Gaussian acceleration structure, intersection shader (collect, sort, accumulate alpha)
Xuan	GPU ray generation, Hybrid routing (bad mesh areas → Gaussians), one output for all modes, to PyTorch
Jing	Export Gaussians, Python reference, per-Gaussian alpha math, accuracy evaluation